TÉCNICO
UNIVERSIDADE
DE LISBOA



ADEPT: Adaptive Defense via Ensemble and Purification Transformer

Guilherme Silvério de Carvalho Romeiro Leitão

Thesis to obtain the Master of Science Degree in

Computer Science and Engineering

Supervisor: Prof. Fernando Manuel Valente Ramos

October 2026

Declaration
I declare that this document is an original work of my own authorship and that it fulfills all the requirements of the Code of Conduct and Good Practices of the Universidade de Lisboa.

Acknowledgments

I would like to thank my parents for their friendship, encouragement and caring over all these years, for always being there for me through thick and thin and without whom this project would not be possible. I would also like to thank my grandparents, aunts, uncles and cousins for their understanding and support throughout all these years.

Quisque facilisis erat a dui. Nam malesuada ornare dolor. Cras gravida, diam sit amet rhoncus ornare, erat elit consectetuer erat, id egestas pede nibh eget odio. Proin tincidunt, velit vel porta elementum, magna diam molestie sapien, non aliquet massa pede eu diam. Aliquam iaculis.

Fusce et ipsum et nulla tristique facilisis. Donec eget sem sit amet ligula viverra gravida. Etiam vehicula urna vel turpis. Suspendisse sagittis ante a urna. Morbi a est quis orci consequat rutrum. Nullam egestas feugiat felis. Integer adipiscing semper ligula. Nunc molestie, nisl sit amet cursus convallis, sapien lectus pretium metus, vitae pretium enim wisi id lectus.

Donec vestibulum. Etiam vel nibh. Nulla facilisi. Mauris pharetra. Donec augue. Fusce ultrices, neque id dignissim ultrices, tellus mauris dictum elit, vel lacinia enim metus eu nunc.

I would also like to acknowledge my dissertation supervisors Prof. Some Name and Prof. Some Other Name for their insight, support and sharing of knowledge that has made this Thesis possible.

Last but not least, to all my friends and colleagues that helped me grow as a person and were always there for me during the good and bad times in my life. Thank you.

To each and every one of you – Thank you.

Abstract

Nulla facilisi. In vel sem. Morbi id urna in diam dignissim feugiat. Proin molestie tortor eu velit. Aliquam erat volutpat. Nullam ultrices, diam tempus vulputate egestas, eros pede varius leo, sed imperdiet lectus est ornare odio. Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Proin consectetuer velit in dui. Phasellus wisi purus, interdum vitae, rutrum accumsan, viverra in, velit. Sed enim risus, congue non, tristique in, commodo eu, metus. Aenean tortor mi, imperdiet id, gravida eu, posuere eu, felis. Mauris sollicitudin, turpis in hendrerit sodales, lectus ipsum pellentesque ligula, sit amet scelerisque urna nibh ut arcu. Aliquam in lacus. Vestibulum ante ipsum primis in faucibus orci luctus et ultrices posuere cubilia Curae; Nulla placerat aliquam wisi. Mauris viverra odio. Quisque fermentum pulvinar odio. Proin posuere est vitae ligula. Etiam euismod. Cras a eros.

Keywords

Maecenas tempus dictum libero; Donec non tortor in arcu mollis feugiat;Cras rutrum pulvinar tellus.

Resumo

Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Vestibulum tortor quam, feugiat vitae, ultricies eget, tempor sit amet, ante. Donec eu libero sit amet quam egestas semper. Aenean ultricies mi vitae est. Mauris placerat eleifend leo. Quisque sit amet est et sapien ullamcorper pharetra. Vestibulum erat wisi, condimentum sed, commodo vitae, ornare sit amet, wisi. Aenean fermentum, elit eget tincidunt condimentum, eros ipsum rutrum orci, sagittis tempus lacus enim ac dui. Donec non enim in turpis pulvinar facilisis. Ut felis. Aliquam aliquet, est a ullamcorper condimentum, tellus nulla fringilla elit, a iaculis nulla turpis sed wisi. Fusce volutpat. Etiam sodales ante id nunc. Proin ornare dignissim lacus. Nunc porttitor nunc a sem. Sed sollicitudin velit eu magna. Aliquam erat volutpat. Vivamus ornare est non wisi. Proin vel quam. Vivamus egestas. Nunc tempor diam vehicula mauris. Nullam sapien eros, facilisis vel, eleifend non, auctor dapibus, pede.

Palavras Chave

Colaborativo; Codificaçãoo; Conteúdo Multimédia; Comunicação;

Contents

1 Introduction **1**

1.1 Context and Motivation . . . 1

1.2 Problem Statement . . . 2

1.3 Work Objectives and Challenges . . . 2

1.4 Rationale and Contributions . . . 2

1.4.1 Rationale . . . 2

1.4.2 Contributions . . . 3

1.5 Organization of the Document . . . 3

2 Background **5**

2.1 Machine Learning for NIDS . . . 5

2.1.1 Learning Paradigms . . . 5

2.1.2 Feature Engineering and Representation . . . 6

2.1.3 Model Architectures Relevant to ADEPT . . . 6

2.2 Adversarial Machine Learning . . . 6

2.2.1 Threat Taxonomy . . . 6

2.2.2 Defense Paradigms . . . 6

2.2.3 Realizability and Protocol Constraints . . . 6

2.3 Ensemble Learning for Robustness . . . 7

2.3.1 Bagging, Boosting, Stacking . . . 7

2.3.2 Gradient Masking and Transferability . . . 7

2.4 Reinforcement Learning Fundamentals . . . 7

2.5 Summary . . . 7

3 Related Work **9**

3.1 Adversarial Attacks against ML-based NIDS . . . 9

3.2 Training-time Defenses . . . 10

3.3 Inference-time / Purification Defenses . . . 10

3.4 Reinforcement Learning for Adaptive Defense . . . 10

3.5 Positioning of ADEPT and Gap Analysis . . . 10

4 Solution: ADEPT Architecture **13**

4.1 Threat Model . . . 13
4.2 System Overview . . . 14
4.3 Dataset Construction and Packet-to-Tensor Pipeline . . . 14
4.4 Primary Detection: Heterogeneous Stacking Ensemble . . . 14
4.4.1 Random Forest Base Learner . . . 14
4.4.2 Temporal Convolutional Network Base Learner . . . 15
4.4.3 Attention-Based Meta-Learner (the “Mini-Transformer”) . . . 15
4.5 Input Purification: Reconstructor Transformer . . . 15
4.6 Continuous Adaptation: Reinforcement Learning Agent . . . 16
4.7 Implementation and Tools . . . 16

5 Evaluation **17**

5.1 Research Questions . . . 17
5.2 Experimental Setup . . . 18
5.2.1 Datasets and Threat Generation . . . 18
5.2.2 Implementation Environment . . . 18
5.3 Metrics . . . 18
5.4 Base Learner Results . . . 19
5.5 Meta-Learner Results . . . 19
5.6 Purification Results . . . 19
5.7 Reinforcement Learning Adaptation Results . . . 19
5.8 End-to-End System Evaluation . . . 20
5.9 Discussion and Limitations . . . 20

6 Conclusion **21**

6.1 Summary of Contributions . . . 21
6.2 Answers to Research Questions . . . 21
6.3 System Limitations . . . 22
6.4 Future Work . . . 22

Bibliography **22**

A Code of Project **25**

B A Large Table **31**

List of Figures

List of Tables

B.1 Example table 32
B.2 Example of a very long table spreading in several pages 32
B.3 Sample Table. 34

List of Algorithms

Listagens

A.1 Example of a XML file. 25
A.2 `Matlab Function` 26
A.3 `function.m` 27
A.4 HTML with CSS Code 27
A.5 HTML CSS Javascript Code 29
A.6 PYTHON Code 30

Acronyms

1

Introduction

Contents

1.1 Context and Motivation 1
1.2 Problem Statement 2
1.3 Work Objectives and Challenges 2
1.4 Rationale and Contributions 2
1.5 Organization of the Document 3

1.1 Context and Motivation

- ML-based NIDS now outperform signature-based systems on benchmark datasets (CIC-IDS2017, CIC-DDoS-2019, Kitsune) but inherit a new attack surface: adversarial evasion.
- Real attackers are **black-box** (query-only) and **protocol-constrained** — the Inverse Feature Mapping (IFM) problem: a feature-space perturbation must correspond to a realizable packet/flow.
- Most prior defenses assume unrealistic threat models (white-box gradients) or train against feature-space perturbations that cannot exist as real traffic.

- TANTGen (sibling thesis) closes that gap on the attack side — a black-box, protocol-compliant, Transformer-based adversarial traffic generator. ADEPT is the defense answer, trained and evaluated against exactly that realistic threat, not a synthetic one.

1.2 Problem Statement

- Given a streaming flow of packets that may be benign, naturally malicious (e.g. a DDoS), or adversarially perturbed (TANTGen-style evasion of an upstream NIDS): detect the adversarial manipulation **as early as possible** and **neutralize** it before it reaches the protected NIDS — without degrading clean-traffic accuracy.
- Key constraints: streaming (no full-flow buffering), black-box realism, protocol compliance of any "fixed" output, and adaptability to a generator that itself evolves.

1.3 Work Objectives and Challenges

- O1 — Feature representation: a packet/flow representation usable by heterogeneous learners at different temporal granularities (nPrint bit-level + IPAT + Direction).
- O2 — Ensemble-based detection: combine architecturally diverse base learners (differentiable vs. non-differentiable) to raise the cost of transferable gradient attacks.
- O3 — Input purification: reconstruct a "denoised" version of a flagged flow that lands back on the benign manifold while remaining a valid, realizable packet sequence.
- O4 — Continuous adaptation: a reinforcement-learning component closes the loop so the defense keeps adapting to an evolving attacker rather than staying static.
- O5 — Comprehensive, streaming-aware evaluation (not just end-of-flow accuracy).
- Challenges: extreme class/source imbalance (reflection floods dominate flow counts); near-invisibility of pure-timing perturbations (`iat` mode); latency budget at 100 Gbps+.

1.4 Rationale and Contributions

1.4.1 Rationale

- Why stacking over a single architecture: gradient-based perturbations crafted against a differentiable model transfer poorly to a non-differentiable one (RF) — a "rugged" decision boundary.

- Why a Transformer purifier over GAN/AutoEncoder: causal/sequential structure of packet streams; KV-cache-friendly decoding for latency.
- Why an adaptive (RL) component over a static defense: static defenses are eventually learned and bypassed; an adaptive loop lets the defense keep moving.

1.4.2 Contributions

- A streaming-native packet-to-tensor pipeline unifying dense (TCN) and sparse (RF) prediction cadences over the same nPrint-based representation.
- A heterogeneous RF+TCN ensemble fused by an attention-based meta-learner, evaluated on a like-for-like streaming axis (AUROC@t, EDP), not just end-of-flow metrics.
- A causal purification Transformer achieving (measured) a 7.5$\times$ MSE reduction and a 0.000 residual detection gap on strongly-perturbed sources, with `tshark`-verified protocol compliance on realized PCAPs.
- A reinforcement-learning feedback loop that lets the defense adapt online to an evolving adversary.
- A 10-source, 3-perturbation-mode evaluation corpus (Kitsune + CIC-IDS2017 + CIC-DDoS-2019 $\times$ TANTGen) reusable for future ADEPT-class defense work.

1.5 Organization of the Document

- One paragraph per chapter roadmap (Background, Related Work, Solution, Evaluation, Conclusion).

2

Background

Contents

2.1 Machine Learning for NIDS 5

2.2 Adversarial Machine Learning 6

2.3 Ensemble Learning for Robustness 7

2.4 Reinforcement Learning Fundamentals 7

2.5 Summary 7

2.1 Machine Learning for NIDS

2.1.1 Learning Paradigms

- Supervised (RF/TCN/meta-learner case), unsupervised (Kitsune autoencoders), semi-supervised, reinforcement learning — when each is appropriate; ADEPT uses three of the four.

2.1.2 Feature Engineering and Representation

- Static vs. dynamic features; packet- vs. flow-level granularity (traffic-space vs. feature-space).
- nPrint bit-level encoding — why bit-level beats hand-crafted headers for cross-protocol generality; identity-field masking (src/dst IP, ports, IP ID) to prevent overfitting to flow identity instead of behavior.
- Multi-rate aggregation: per-packet sequence (TCN) vs. prefix-checkpoint statistics at packet 2/10/20/... (RF), motivated by PortScan's $\sim$2-packet average flow length.

2.1.3 Model Architectures Relevant to ADEPT

- Random Forests: bagging, discrete non-differentiable boundaries.
- 1D-CNN / Temporal Convolutional Networks: causal dilated convolutions; receptive-field math (29 packets for 3 blocks at dilation $\{2, 4, 8\}$).
- Transformers: self-attention for sequence modeling; decoder-only vs. encoder-only trade-offs; KV-caching for streaming inference latency.

2.2 Adversarial Machine Learning

2.2.1 Threat Taxonomy

- Attacker knowledge: white-/gray-/black-box. Attacker goal: poisoning vs. evasion.
- The Inverse Feature Mapping (IFM) problem — why network AML is harder than image AML.

2.2.2 Defense Paradigms

- Adversarial training, input transformation/purification, defensive distillation, certified robustness (randomized smoothing, Lipschitz bounds) — strengths/limits of each for traffic data.

2.2.3 Realizability and Protocol Constraints

- Why a "valid feature vector" $\neq$ a "valid packet"; checksum/flag/length consistency.
- The realization step (feature-space $\rightarrow$ traffic-space) as a first-class design concern, not an afterthought.

2.3 Ensemble Learning for Robustness

2.3.1 Bagging, Boosting, Stacking

- Mechanics of each; why stacking with an attention meta-learner suits heterogeneous base learners better than averaging/voting.

2.3.2 Gradient Masking and Transferability

- Argument for why a non-differentiable RF degrades the transferability of a white-box-crafted PGD/FGSM perturbation optimized against a differentiable model.

2.4 Reinforcement Learning Fundamentals

- Generic RL primer at the level needed to motivate an adaptive defense: agent/environment, state/action/reward, policy — kept general, no ADEPT-specific instantiation here.
- Concept of RL for adaptive/online defense (literature surveyed in Chapter 3); no commitment to a specific algorithm or formulation.

2.5 Summary

- Bridge: representation, ensembling, purification, and RL are the four pillars composed in ADEPT (Chapter 4).

3

Related Work

Contents

3.1 Adversarial Attacks against ML-based NIDS 9
3.2 Training-time Defenses 10
3.3 Inference-time / Purification Defenses 10
3.4 Reinforcement Learning for Adaptive Defense 10
3.5 Positioning of ADEPT and Gap Analysis 10

3.1 Adversarial Attacks against ML-based NIDS

- Feature-space attacks (gradient-based: FGSM/PGD); the Tiki-Taka result (35.7% evasion against SOTA DL NIDS) as the “why CNN-only is risky” data point.

- Traffic-space / realizable attacks: GPMT, PANTS (SMT-solver realizability), DeepRed (GAN C2 traffic), and **TANTGen** (direct sibling — Transformer, black-box, protocol-compliant; treated in depth, as it is this thesis’s actual attacker).

3.2 Training-time Defenses

- Statistical sanitization / poisoning defense (Antidote; bagging as a form of certified robustness).
- Federated/split-learning defenses (FedTGE, AlignIns, CrowdGuard) — brief, lower relevance.
- Backdoor unlearning (SEAM, BayBFed, PBP) — brief.
- Realizable adversarial training (PANTS, DeepRed) — the IFM-aware training line ADEPT's dataset construction follows in spirit (training on realizable adversarials, not synthetic feature perturbations).

3.3 Inference-time / Purification Defenses

- Certified robustness (CertTA randomized smoothing; LipsLev edit-distance Lipschitz bounds).
- Generative/reconstruction defenses: Defense-GAN (origin of "project onto the benign manifold"), AdvPurRec (diffusion), GTAE-IDS (Transformer autoencoder), DiffuseDef (latent diffusion), CausalDiff, ADBM — position ADEPT's reconstructor against each (causal decoder-only, residual/$\Delta \approx 0$-by-default, not diffusion-based — a deliberate latency/simplicity trade-off).
- Constraint-based detection (NIDS-CBAD) as a lightweight alternative ADEPT does not adopt, and why.

3.4 Reinforcement Learning for Adaptive Defense

- Survey prior RL-for-defense work at a high level (e.g. ARCS, PRO, Slot, CAMP) — literature only, no claims about ADEPT's own mechanism.
- Identified gap: no prior framework couples generative purification with RL-driven continuous adaptation in one closed loop — ADEPT positions itself in that gap.

3.5 Positioning of ADEPT and Gap Analysis

- Comparison table of surveyed defenses (phase, scope, techniques) with ADEPT's actual measured row (not aspirational).
- Of the surveyed works, none combine (a) realizable black-box training data, (b) heterogeneous-architecture ensembling for transferability resistance, (c) generative purification, and (d) online

RL adaptation — ADEPT's contribution is the combination, evaluated end-to-end on a 10-source corpus rather than a single dataset.

4

Solution: ADEPT Architecture

Contents

4.1 Threat Model 13
4.2 System Overview 14
4.3 Dataset Construction and Packet-to-Tensor Pipeline 14
4.4 Primary Detection: Heterogeneous Stacking Ensemble 14
4.5 Input Purification: Reconstructor Transformer 15
4.6 Continuous Adaptation: Reinforcement Learning Agent 16
4.7 Implementation and Tools 16

4.1 Threat Model

- Black-box evasion attacker (query/label access only); functional-realizability assumption — an unrealizable attack (broken checksum, negative IAT) is out of scope by design.
- Attacker goal: evade the protected downstream NIDS. ADEPT's goal: detect + purify before that NIDS ever sees the manipulated flow.

- Out of scope (non-goals, by design): poisoning, white-box gradient attacks, and non-protocol-compliant attacks.

4.2 System Overview

- Pipeline: Packet-to-Tensor $\rightarrow$ {TCN, RF} $\rightarrow$ Mini-Transformer meta-learner $\rightarrow$ (if adversarial) Reconstructor $\rightarrow$ Protected NIDS.
- The RL agent sits outside this critical path, observing outcomes and adapting the defense over time.
- Two phases: Training (fit the ensemble, meta-learner, and purifier) and Deployment (the closed-loop detect/purify/adapt cycle).

4.3 Dataset Construction and Packet-to-Tensor Pipeline

- Sources: Kitsune (Mirai, SSDP Flood), CIC-IDS2017 (DDoS, PortScan, Brute Force), CIC-DDoS-2019 (NTP/DNS/SSDP/SYN/LDAP reflection floods) $\times$ TANTGen adversarial variants (`both`/`iat`/`size`).
- Label convention: raw-dataset traffic = benign (upstream NIDS's job); TANTGen-generated = adversarial.
- nPrint bit-level per-packet encoding (1088 bits) + IPAT (z-scored) + Direction = 1090-dim token; identity-field masking (src/dst IP, ports, IP ID).
- Dual emission cadence: per-packet (`flow_packets.npy`) for the TCN; multi-scale prefix checkpoints at packet 2 then every 10 (`rf_checkpoints.npy`, 24-dim Welford-aggregated stats) for the RF — PortScan's $\sim$2-packet average needs an early RF checkpoint or it never gets an RF verdict.
- Benign-cap ratio (3:1 vs. adversarial) to stop the reflection-flood-dominated benign pool from swamping training; flow-atomic, source-stratified 70/15/15 split, seed 42, shared across all downstream models for fair comparison.

4.4 Primary Detection: Heterogeneous Stacking Ensemble

4.4.1 Random Forest Base Learner

- 24-dim aggregated prefix statistics (mean/std/min/max over IPAT, packet size, payload length/entropy, TCP flags, direction); grid-searched config (200 trees, depth 20, min_leaf 5, balanced_subsample).

- Rationale: discrete, non-differentiable boundary $\Rightarrow$ gradient masking against white-box-style perturbations even under the stated black-box threat model.
- Measured: ROC AUC 0.888 (per-checkpoint), Recall-adv 0.982, EDP median 2 packets, detection rate 99.69%.

4.4.2 Temporal Convolutional Network Base Learner

- Causal dilated TemporalBlocks $[128, 128, 128]$, $k = 3$, receptive field 29 packets, 805K params; per-timestep $[B, T, 2]$ logits — dense, early-warning stream (verdict from packet 1).
- Measured: ROC AUC 0.836, EDP median 1 packet, detection rate 86.31%.

4.4.3 Attention-Based Meta-Learner (the "Mini-Transformer")

- 13-dim per-packet fused token (carry-forward RF logit, dense TCN logit, rf_age / is_checkpoint / IPAT / direction, compact packet summary); causal mask; learnable `rf_unknown_logit` for pre-checkpoint NaN slots; 68K params, 2 layers / 4 heads / $d_{\text{model}} = 64$.
- "Beat the best base" auxiliary loss (weight 0.5): shrinks the training confidence gap $\sim 9\times$ but does not move test AUROC@t (rank-bottlenecked, not loss-bottlenecked) — reported honestly.
- Measured: matches/beats both base learners on the flow-level streaming axis from packet 5 onward (AUROC@5 0.834 $\rightarrow$ @max 0.845) while keeping the TCN's EDP$= 1$ latency.
- Discussion: why RF "wins" on its own per-checkpoint axis but not on the comparable flow-level axis (avoid the apples-to-oranges trap).

4.5 Input Purification: Reconstructor Transformer

- Causal Transformer encoder + residual head ($\Delta \approx 0$ default), 3 layers / 4 heads / $d_{\text{model}} = 128$, 678K params; trained on 1.99M 5-tuple-matched (adversarial $\rightarrow$ clean) pairs; smooth-L1, masked over padding, length-normalized per flow.
- Decoder-only/causal choice over GAN/AutoEncoder justified by KV-cache streaming latency.
- Loss-design choice: `loss_emphasis: none` (uniform-weight reconstruction, no attack-footprint hint) as the **main, attack-agnostic result**; the perturbation-weighted variant is an explicit off-by-default ablation.

- Realization step: maps purified feature deltas back onto real PCAP packets; `tshark`-validated protocol compliance (produce/validate real packets, not just tensors).
- Measured: MSE 0.962 $\rightarrow$ 0.128 (7.5$\times$); purification gain 0.128 with **residual gap 0.000** (purified flow statistically indistinguishable from genuinely clean to the detector); 99.99% feature-space / 99.5% `tshark` packet compliance.
- Attack-dependent effect: strong on `portscan`/`ssdp_flood`/`ddos` (`both`/`size`, gain 0.86–0.96); $\sim$0 on pure-`iat` and the four hardest reflection floods (DNS/NTP/SSDP/LDAP) — reported as a limitation, not buried.

4.6 Continuous Adaptation: Reinforcement Learning Agent

- Role in the pipeline: an RL agent sits outside the critical path, observes detection outcomes, and adapts the defense over time so it tracks an evolving attacker (high-level only).
- To be specified once the agent is designed: the learning formulation, what it observes, what it controls, and how the feedback loop closes — left unspecified here on purpose.

4.7 Implementation and Tools

- PyTorch (TCN / Transformer / meta-learner / reconstructor), scikit-learn (RF), Stable-Baselines3 (RL agent), Scapy + tshark/editcap/tcpdump (PCAP handling, realization, compliance checks), nPrint (bit-level encoding).
- Repository-layout walkthrough (config/, data/, scripts/, src/) — brief, points to the README for detail.

5

Evaluation

Contents

5.1 Research Questions 17
5.2 Experimental Setup 18
5.3 Metrics 18
5.4 Base Learner Results 19
5.5 Meta-Learner Results 19
5.6 Purification Results 19
5.7 Reinforcement Learning Adaptation Results 19
5.8 End-to-End System Evaluation 20
5.9 Discussion and Limitations 20

5.1 Research Questions

- RQ1 — Ensemble robustness: does heterogeneous stacking measurably limit gradient-attack transfer vs. a single-architecture baseline?

- RQ2 — Purification effectiveness: does the reconstructor close the detection gap while staying protocol-compliant?
- RQ3 — Adaptability: does the adaptive (RL) component let the defense keep up with an evolving attacker better than a static baseline? (phrased generically — no metric committed)
- RQ4 — Latency/overhead: does the full pipeline (incl. Transformer purification) stay within a real-time-NIDS-compatible latency budget?

5.2 Experimental Setup

5.2.1 Datasets and Threat Generation

- 10 benign sources $\times$ 3 TANTGen perturbation modes (`both`/`iat`/`size`); flow/packet counts table.
- TANTGen as the red-team generator, treated as a black box by ADEPT.

5.2.2 Implementation Environment

- Hardware (e.g. 2$\times$ Xeon Gold 5320, 192 GB RAM, 4$\times$ RTX A4000) — confirm against the actual run.
- Software stack recap (cross-ref Section 4.7).

5.3 Metrics

- Classification & robustness: Macro F1, ROC/PR AUC, Recall(adversarial), Attack Success Rate (ASR).
- Streaming-specific: AUROC@t (per checkpoint t), EDP (Earliest Detection Packet, median).
- Purification: MSE before/after, purification gain ($\Delta P(\text{adv})$ through the protected NIDS), residual gap, feature-space invariant compliance, `tshark` protocol compliance.
- Adaptation: placeholder for adaptation metrics vs. a static baseline (to be defined with the agent).
- Efficiency: throughput (pps), inference latency (ms) with purification-overhead breakout, GPU/CPU utilization.

5.4 Base Learner Results

- RF table (ROC AUC 0.888, PR AUC 0.937, Macro F1 0.838, Recall-adv 0.982, EDP 2, DR 99.69%).
- TCN table (ROC AUC 0.836, PR AUC 0.943, Macro F1 0.726, Recall-adv 0.857, EDP 1, DR 86.31%).
- Per-source breakdown: easy sources (`portscan`, `ddos`, `ddos_kitsune`, `ssdp_flood` $\geq$0.99 on `both`/`size`) vs. hard reflection floods (DNS/NTP/SSDP/LDAP $\approx$0.52–0.62) vs. moderate (`syn_ddos` $\approx$0.88–0.90).
- Cross-mode breakdown: `iat`-only consistently hardest (e.g. `portscan_iat` TCN AUROC@max 0.31).

5.5 Meta-Learner Results

- Streaming comparison table (RF vs. TCN vs. Meta; AUROC@1/2/5/10/max, EDP, detection rate).
- Discuss the "RF wins per-checkpoint but Meta wins flow-level from packet 5" nuance to preempt the "why is RF AUC higher" question.
- Ablation note on the auxiliary loss (confidence-gap reduction without AUROC movement).

5.6 Purification Results

- Reconstruction / purification-gain / compliance table; strongly-purified vs. unaffected-source breakdown.
- Caveats (own-TCN-as-oracle limitation; tshark sample cap; the two tiny `ssdp_flood` samples hitting 0% compliance due to snap-length truncation — explain, don't hide).
- Optional: a qualitative before/after example (one packet trace) if space/figures allow.

5.7 Reinforcement Learning Adaptation Results

- Placeholder section: results of the adaptive component vs. a static baseline — to be defined once the agent exists. No specific experiments, curves, or numbers committed here.

5.8 End-to-End System Evaluation

- Full-pipeline Attack Success Rate before vs. after ADEPT, across all 10×3 source/mode combinations — the headline table.
- Latency/throughput results (RQ4): purification-overhead breakout; whether the system meets a stated real-time budget; resource utilization.

5.9 Discussion and Limitations

- Honest accounting: hard reflection floods and `iat`-only perturbations remain weak points for detection and purification (the feature signal is genuinely thin, not a modeling failure).
- Purification measured through ADEPT's own TCN — flag as a limitation (independent-NIDS validation is future work).
- Any limitations of the adaptive component — to be filled once it is designed.

6

Conclusion

Contents

6.1 Summary of Contributions 21
6.2 Answers to Research Questions 21
6.3 System Limitations 22
6.4 Future Work 22

6.1 Summary of Contributions

- Recap the five contributions from Section 1.4 in completed tense, each cross-referencing the chapter/table that substantiates it.

6.2 Answers to Research Questions

- One paragraph per RQ (Section 5.1), each closing with the evidence (table/figure ref) that answers it.

6.3 System Limitations

- Hard reflection-flood sources and IAT-only perturbations (real, measured limitation).
- Purification evaluated through an own-TCN oracle, not an independent NIDS.
- Any limitation specific to the adaptive component — to be filled in later.
- Feature-space scope: nPrint + IPAT + Direction; no payload/DPI features (relevant given the TLS/QUIC encryption trend).

6.4 Future Work

- Independent-NIDS purification validation (closing the own-oracle limitation).
- Maturing/extending the adaptive (RL) component.
- Feature-space expansion / payload-aware representations for encrypted-traffic robustness.
- Real-time deployment study at line rate (100 Gbps+) beyond the simulated latency benchmark.

Bibliography

A

Code of Project

Nulla dui purus, eleifend vel, consequat non, dictum porta, nulla. Duis ante mi, laoreet ut, commodo eleifend, cursus nec, lorem. Aenean eu est. Etiam imperdiet turpis. Praesent nec augue. Curabitur ligula quam, rutrum id, tempor sed, consequat ac, dui. Vestibulum accumsan eros nec magna. Vestibulum vitae dui. Vestibulum nec ligula et lorem consequat ullamcorper.

Listagem A.1: Example of a XML file.

```
1  <?xml version="1.0" encoding="UTF-8"?>
2  <StreamInfo version="2.0">
3      <Clip duration="PT01M0.00S">
4          <BaseURL>videos/</BaseURL>
5          <Description>svc_1</Description>
6          <Representation mimeType="video/SVC" codecs="svc" frameRate="30.00" bandwidth="401.90"
7              width="176" height="144" id="L0">
8              <BaseURL>svc_1/</BaseURL>
9              <SegmentInfo from="0" to="11" duration="PT5.00S">
10                 <BaseURL>svc_1-L0-</BaseURL>
11             </SegmentInfo>
12         </Representation>
```

```
13          <Representation mimeType="video/SVC" codecs="svc" frameRate="30.00" bandwidth="1322.60"
14              width="352" height="288" id="L1">
15              <BaseURL>svc_1/</BaseURL>
16              <SegmentInfo from="0" to="11" duration="PT5.00S">
17                  <BaseURL>svc_1-L1-</BaseURL>
18              </SegmentInfo>
19          </Representation>
20      </Clip>
21  </StreamInfo>
```

Etiam imperdiet turpis. Praesent nec augue. Curabitur ligula quam, rutrum id, tempor sed, consequat ac, dui. Maecenas tincidunt velit quis orci. Sed in dui. Nullam ut mauris eu mi mollis luctus. Class aptent taciti sociosqu ad litora torquent per conubia nostra, per inceptos hymenaeos. Sed cursus cursus velit. Sed a massa. Duis dignissim euismod quam.

Class aptent taciti sociosqu ad litora torquent per conubia nostra, per inceptos hymenaeos. Phasellus eget nisl ut elit porta ullamcorper. Maecenas tincidunt velit quis orci. Sed in dui. Nullam ut mauris eu mi mollis luctus. Class aptent taciti sociosqu ad litora torquent per conubia nostra, per inceptos hymenaeos.

This inline MATLAB code `for i=1:3, disp('cool'); end;` uses the `\mcode{}` command.[1]

Nullam ut mauris eu mi mollis luctus. Class aptent taciti sociosqu ad litora torquent per conubia nostra, per inceptos hymenaeos. Sed cursus cursus velit. Sed a massa. Duis dignissim euismod quam. Nullam euismod metus ut orci.

Listagem A.2: `Matlab Function`

```
1  for i = 1:3
2      if i >= 5 && a ~= b        % literate programming replacement
3          disp('cool');          % comment with some LaTeXin it:  πx^2
4      end
5      [:,ind] = max(vec);
6      x_last = x(1,end) - 1;
7      v(end);
8      ylabel('Voltage (μV)');
9  end
```

Nullam ut mauris eu mi mollis luctus. Class aptent taciti sociosqu ad litora torquent per conubia nostra, per inceptos hymenaeos. Sed cursus cursus velit. Sed a massa. Duis dignissim euismod quam. Nullam euismod metus ut orci.

[1] MATLAB Works also in footnotes: `for i=1:3, disp('cool'); end;`

Listagem A.3: `function.m`

```
%   Copyright 2010 The MathWorks, Inc.
function ObjTrack(position)
% #codegen
% First, setup the figure
numPts = 300;              % Process and plot 300 samples
figure;hold;grid;          % Prepare plot window
% Main loop
for idx = 1: numPts
    z = position(:,idx);   % Get the input data
    y = kalmanfilter(z);   % Call Kalman filter to estimate the position
    plot_trajectory(z,y);  % Plot the results
end
hold;
end   % of the function
```

Class aptent taciti sociosqu ad litora torquent per conubia nostra, per inceptos hymenaeos. Phasellus eget nisl ut elit porta ullamcorper. Maecenas tincidunt velit quis orci. Sed in dui. Nullam ut mauris eu mi mollis luctus. Class aptent taciti sociosqu ad litora torquent per conubia nostra, per inceptos hymenaeos. Sed cursus cursus velit. Sed a massa. Duis dignissim euismod quam. Nullam euismod metus ut orci. Vestibulum erat libero, scelerisque et, porttitor et, varius a, leo.

Listagem A.4: HTML with CSS Code

```
<!DOCTYPE html>
<html>
  <head>
    <title>Listings Style Test</title>
    <meta charset="UTF-8">
    <style>
      /* CSS Test */
      * {
        padding: 0;
        border: 0;
        margin: 0;
      }
    </style>
    <link rel="stylesheet" href="css/style.css" />
  </head>
```

```
  <header> hey </header>
  <article> this is a article </article>
  <body>
    <!-- Paragraphs are fine -->
    <div id="box">
      <p>
        Hello World
      </p>
      <p>Hello World</p>
      <p id="test">Hello World</p>
      <p></p>
    </div>
    <div>Test</div>
    <!-- HTML script is not consistent -->
    <script src="js/benchmark.js"></script>
    <script>
      function createSquare(x, y) {
        // This is a comment.
        var square = document.createElement('div');
        square.style.width = square.style.height = '50px';
        square.style.backgroundColor = 'blue';

        /*
         * This is another comment.
         */
        square.style.position = 'absolute';
        square.style.left = x + 'px';
        square.style.top = y + 'px';

        var body = document.getElementsByTagName('body')[0];
        body.appendChild(square);
      };

      // Please take a look at +=
      window.addEventListener('mousedown', function(event) {
        // German umlaut test: Berührungspunkt ermitteln
        var x = event.touches[0].pageX;
        var y = event.touches[0].pageY;
```

```
54          var lookAtThis += 1;
55        });
56      </script>
57    </body>
58  </html>
```

Nulla dui purus, eleifend vel, consequat non, dictum porta, nulla. Duis ante mi, laoreet ut, commodo eleifend, cursus nec, lorem. Aenean eu est. Etiam imperdiet turpis. Praesent nec augue. Curabitur ligula quam, rutrum id, tempor sed, consequat ac, dui. Vestibulum accumsan eros nec magna. Vestibulum vitae dui. Vestibulum nec ligula et lorem consequat ullamcorper.

Listagem A.5: HTML CSS Javascript Code

```
1
2  @media only screen and (min-width: 768px) and (max-width: 991px) {
3
4    #main {
5      width: 712px;
6      padding: 100px 28px 120px;
7    }
8
9    /* .mono {
10     font-size: 90%;
11   } */
12
13   .cssbtn a {
14     margin-top: 10px;
15     margin-bottom: 10px;
16     width: 60px;
17     height: 60px;
18     font-size: 28px;
19     line-height: 62px;
20   }
```

Nulla dui purus, eleifend vel, consequat non, dictum porta, nulla. Duis ante mi, laoreet ut, commodo eleifend, cursus nec, lorem. Aenean eu est. Etiam imperdiet turpis. Praesent nec augue. Curabitur ligula quam, rutrum id, tempor sed, consequat ac, dui. Vestibulum accumsan eros nec magna. Vestibulum vitae dui. Vestibulum nec ligula et lorem consequat ullamcorper.

Listagem A.6: PYTHON Code

```
class TelgramRequestHandler(object):
    def handle(self):
        addr = self.client_address[0]          # Client IP-adress
        telgram = self.request.recv(1024)      # Recieve telgram
        print "From: %s, Received: %s" % (addr, telgram)
        return
```

B

A Large Table

Aliquam et nisl vel ligula consectetuer suscipit. Morbi euismod enim eget neque. Donec sagittis massa. Vestibulum quis augue sit amet ipsum laoreet pretium. Nulla facilisi. Duis tincidunt, felis et luctus placerat, ipsum libero vestibulum sem, vitae elementum wisi ipsum a metus. Nulla a enim sed dui hendrerit lobortis. Donec lacinia vulputate magna. Vivamus suscipit lectus at quam. In lectus est, viverra a, ultricies ut, pulvinar vitae, tellus. Donec et lectus et sem rutrum sodales. Morbi cursus. Aliquam a odio. Sed tortor velit, convallis eget, porta interdum, convallis sed, tortor. Phasellus ac libero a lorem auctor mattis. Lorem ipsum dolor sit amet, consectetuer adipiscing elit.

Nunc auctor bibendum eros. Maecenas porta accumsan mauris. Etiam enim enim, elementum sed, bibendum quis, rhoncus non, metus. Fusce neque dolor, adipiscing sed, consectetuer et, lacinia sit amet, quam. Suspendisse wisi quam, consectetuer in, blandit sed, suscipit eu, eros. Etiam ligula enim, tempor ut, blandit nec, mollis eu, lectus. Nam cursus. Vivamus iaculis. Aenean risus purus, pharetra in, blandit quis, gravida a, turpis. Donec nisl. Aenean eget mi. Fusce mattis est id diam. Phasellus faucibus interdum sapien. Duis quis nunc. Sed enim. Nunc auctor bibendum eros. Maecenas porta accumsan mauris. Etiam enim enim, elementum sed, bibendum quis, rhoncus non, metus. Fusce neque dolor, adipiscing sed, consectetuer et, lacinia sit amet, quam.

As Table B.1 shows, the data can be inserted from a file, in the case of a somehow complex structure. Notice the Table footnotes.

Table B.1: Example table

Benchmark: ANN	**#Layers** (1)	**#Nets** (2)	#Nodes* (3) = 8 · (1) · (2)	Critical path (4) = 4 · (1)	**Latency (T_{iter})** (5)
A1	**3–1501**	1	**24–12008**	**12–6004**	4
A2	501	1	4008	2004	**2–2000**
A3	10	**2–1024**	**160–81920**	40	60†
A4	10	50	4000	40	**80–1200**

Benchmark: FFT	**FFT size‡** (1)	*#Inputs* (2) = $2^{(1)}$	*#Nodes** (3) = 10 · (1) · (2)	*Critical path* (4) = 4 · (1)	**Latency (T_{iter})** (5)
F1	**1–10**	2–1024	**20–102400**	4–40	6–60†
F2	**5**	32	1600	20	**40 – 1500**

Benchmark: Random networks	**#Types** (1)	**#Nodes** (2)	*#Networks* (3)	*Critical path* (4)	**Latency (T_{iter})** (5)
R1	3	10–2000	500	*variable*	(4)
R2	3	50	500	*variable*	(4) × [1; · · · ; 20]

* *Excluding constant nodes.*
† *Value kept proportional to the critical path:* $(5) = (4) * 1.5$.
‡ *A size of x corresponds to a 2^x point FFT.*
Values in bold indicate the parameter being varied.

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Morbi commodo, ipsum sed pharetra gravida, orci magna rhoncus neque, id pulvinar odio lorem non turpis. Nullam sit amet enim. Suspendisse id velit vitae ligula volutpat condimentum. Aliquam erat volutpat. Sed quis velit. Nulla facilisi. Nulla libero. Vivamus pharetra posuere sapien. Nam consectetuer. Sed aliquam, nunc eget euismod ullamcorper, lectus nunc ullamcorper orci, fermentum bibendum enim nibh eget ipsum. Donec porttitor ligula eu dolor. Maecenas vitae nulla consequat libero cursus venenatis. Nam magna enim, accumsan eu, blandit sed, blandit a, eros.

And now an example (Table B.2) of a table that extends to more than one page. Notice the repetition of the Caption (with indication that is continued) and of the Header, as well as the continuation text at the bottom.

Table B.2: Example of a very long table spreading in several pages

Time (s)	**Triple chosen**	**Other feasible triples**
0	(1, 11, 13725)	(1, 12, 10980), (1, 13, 8235), (2, 2, 0), (3, 1, 0)
2745	(1, 12, 10980)	(1, 13, 8235), (2, 2, 0), (2, 3, 0), (3, 1, 0)
5490	(1, 12, 13725)	(2, 2, 2745), (2, 3, 0), (3, 1, 0)
8235	(1, 12, 16470)	(1, 13, 13725), (2, 2, 2745), (2, 3, 0), (3, 1, 0)
10980	(1, 12, 16470)	(1, 13, 13725), (2, 2, 2745), (2, 3, 0), (3, 1, 0)
13725	(1, 12, 16470)	(1, 13, 13725), (2, 2, 2745), (2, 3, 0), (3, 1, 0)
16470	(1, 13, 16470)	(2, 2, 2745), (2, 3, 0), (3, 1, 0)

Continued on next page

Table B.2 – continued from previous page

Time (s)	Triple chosen	Other feasible triples
19215	(1, 12, 16470)	(1, 13, 13725), (2, 2, 2745), (2, 3, 0), (3, 1, 0)
21960	(1, 12, 16470)	(1, 13, 13725), (2, 2, 2745), (2, 3, 0), (3, 1, 0)
24705	(1, 12, 16470)	(1, 13, 13725), (2, 2, 2745), (2, 3, 0), (3, 1, 0)
27450	(1, 12, 16470)	(1, 13, 13725), (2, 2, 2745), (2, 3, 0), (3, 1, 0)
30195	(2, 2, 2745)	(2, 3, 0), (3, 1, 0)
32940	(1, 13, 16470)	(2, 2, 2745), (2, 3, 0), (3, 1, 0)
35685	(1, 13, 13725)	(2, 2, 2745), (2, 3, 0), (3, 1, 0)
38430	(1, 13, 10980)	(2, 2, 2745), (2, 3, 0), (3, 1, 0)
41175	(1, 12, 13725)	(1, 13, 10980), (2, 2, 2745), (2, 3, 0), (3, 1, 0)
43920	(1, 13, 10980)	(2, 2, 2745), (2, 3, 0), (3, 1, 0)
46665	(2, 2, 2745)	(2, 3, 0), (3, 1, 0)
49410	(2, 2, 2745)	(2, 3, 0), (3, 1, 0)
52155	(1, 12, 16470)	(1, 13, 13725), (2, 2, 2745), (2, 3, 0), (3, 1, 0)
54900	(1, 13, 13725)	(2, 2, 2745), (2, 3, 0), (3, 1, 0)
57645	(1, 13, 13725)	(2, 2, 2745), (2, 3, 0), (3, 1, 0)
60390	(1, 12, 13725)	(2, 2, 2745), (2, 3, 0), (3, 1, 0)
63135	(1, 13, 16470)	(2, 2, 2745), (2, 3, 0), (3, 1, 0)
65880	(1, 13, 16470)	(2, 2, 2745), (2, 3, 0), (3, 1, 0)
68625	(2, 2, 2745)	(2, 3, 0), (3, 1, 0)
71370	(1, 13, 13725)	(2, 2, 2745), (2, 3, 0), (3, 1, 0)
74115	(1, 12, 13725)	(2, 2, 2745), (2, 3, 0), (3, 1, 0)
76860	(1, 13, 13725)	(2, 2, 2745), (2, 3, 0), (3, 1, 0)
79605	(1, 13, 13725)	(2, 2, 2745), (2, 3, 0), (3, 1, 0)
82350	(1, 12, 13725)	(2, 2, 2745), (2, 3, 0), (3, 1, 0)
85095	(1, 12, 13725)	(1, 13, 10980), (2, 2, 2745), (2, 3, 0), (3, 1, 0)
87840	(1, 13, 16470)	(2, 2, 2745), (2, 3, 0), (3, 1, 0)
90585	(1, 13, 16470)	(2, 2, 2745), (2, 3, 0), (3, 1, 0)
93330	(1, 13, 13725)	(2, 2, 2745), (2, 3, 0), (3, 1, 0)
96075	(1, 13, 16470)	(2, 2, 2745), (2, 3, 0), (3, 1, 0)
98820	(1, 13, 16470)	(2, 2, 2745), (2, 3, 0), (3, 1, 0)
101565	(1, 13, 13725)	(2, 2, 2745), (2, 3, 0), (3, 1, 0)
104310	(1, 13, 16470)	(2, 2, 2745), (2, 3, 0), (3, 1, 0)
107055	(1, 13, 13725)	(2, 2, 2745), (2, 3, 0), (3, 1, 0)
109800	(1, 13, 13725)	(2, 2, 2745), (2, 3, 0), (3, 1, 0)
112545	(1, 12, 16470)	(1, 13, 13725), (2, 2, 2745), (2, 3, 0), (3, 1, 0)
115290	(1, 13, 16470)	(2, 2, 2745), (2, 3, 0), (3, 1, 0)
118035	(1, 13, 13725)	(2, 2, 2745), (2, 3, 0), (3, 1, 0)
120780	(1, 13, 16470)	(2, 2, 2745), (2, 3, 0), (3, 1, 0)
123525	(1, 13, 13725)	(2, 2, 2745), (2, 3, 0), (3, 1, 0)
126270	(1, 12, 16470)	(1, 13, 13725), (2, 2, 2745), (2, 3, 0), (3, 1, 0)
129015	(2, 2, 2745)	(2, 3, 0), (3, 1, 0)
131760	(2, 2, 2745)	(2, 3, 0), (3, 1, 0)
134505	(1, 13, 16470)	(2, 2, 2745), (2, 3, 0), (3, 1, 0)
137250	(1, 13, 13725)	(2, 2, 2745), (2, 3, 0), (3, 1, 0)
139995	(2, 2, 2745)	(2, 3, 0), (3, 1, 0)
142740	(2, 2, 2745)	(2, 3, 0), (3, 1, 0)
145485	(1, 12, 16470)	(1, 13, 13725), (2, 2, 2745), (2, 3, 0), (3, 1, 0)
148230	(2, 2, 2745)	(2, 3, 0), (3, 1, 0)
150975	(1, 13, 16470)	(2, 2, 2745), (2, 3, 0), (3, 1, 0)
153720	(1, 12, 13725)	(2, 2, 2745), (2, 3, 0), (3, 1, 0)

Continued on next page

Table B.2 – continued from previous page

Time (s)	Triple chosen	Other feasible triples
156465	(1, 13, 13725)	(2, 2, 2745), (2, 3, 0), (3, 1, 0)
159210	(1, 13, 13725)	(2, 2, 2745), (2, 3, 0), (3, 1, 0)
161955	(1, 13, 16470)	(2, 2, 2745), (2, 3, 0), (3, 1, 0)
164700	(1, 13, 13725)	(2, 2, 2745), (2, 3, 0), (3, 1, 0)

An example of a large Table that autofits the size to the page margins is illustrated in Table B.3. Please notice the text size that is shrunken in order fot the table to adjust to the page:

Table B.3: Sample Table.

URL	First Time Visit	Last Time Visit	URL Counts	Value	Reference
https://web.facebook.com/	1521241972	1522351859	177	56640	[facebook-2021]
http://localhost/phpmyadmin/	1518413861	1522075694	24	39312	database-management
https://mail.google.com/mail/u/	1516596003	1522352010	36	33264	Google-Gmail-2021
https://github.com/shawon100	1517215489	1522352266	37	27528	Code-Repository
https://www.youtube.com/	1517229227	1521978502	24	14792	Youtube-video-2021